Orchestra

DOSSIER DE LIVRAISON TECHNIQUE

Architecture du prototype SaaS multi-agents IA

Une application démontrable de bout en bout pour qualifier des prospects, piloter des conversations, préparer les rendez-vous, générer des relances et gérer le stock d’un commerce.

VERSION

1.0 — prototype opérationnel

DATE

11 novembre 2026

APPLICATION

orchestra-ai-agency-demo.vercel.app

DÉPÔT

GitHub public

01 — SYNTHÈSE

Ce qui a été livré

Orchestra transforme le brief initial en un produit cohérent plutôt qu'en une collection d'écrans statiques. L'interface Next.js orchestre les parcours, Supabase conserve l'état métier et n8n Manda exécute les tâches IA et les intégrations externes côté serveur.

5 parcours métier

Qualification, conversation, rendez-vous, relance et stock.

3 workflows n8n

Qualification, agent conversationnel et personnalisation des relances.

12 tables isolées

Schéma Supabase préfixé `orchestra_`, avec RLS forcé.

8 tests E2E

Parcours validés sur profils desktop et mobile.

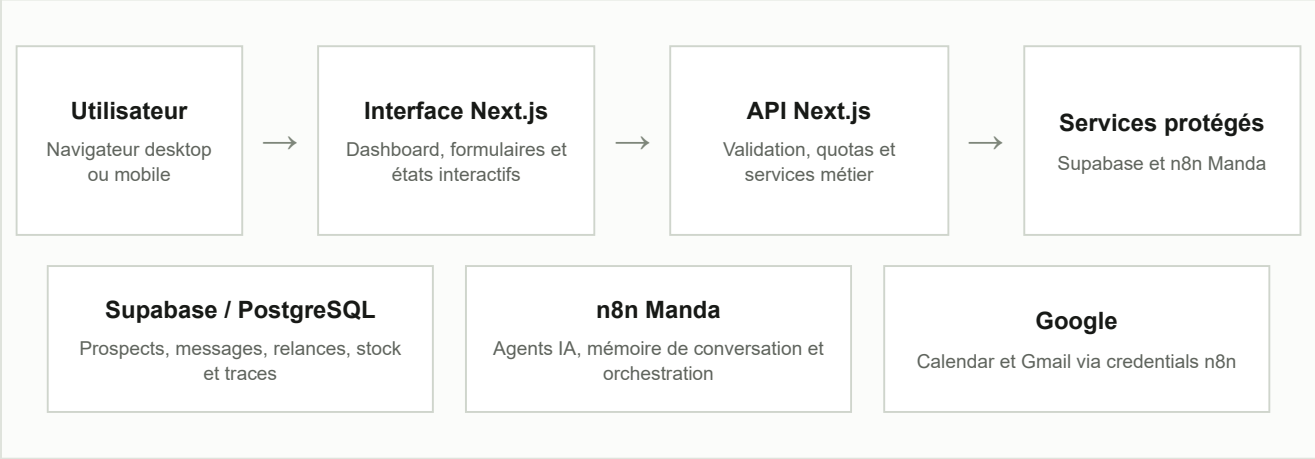
FONCTION	RÉSULTAT LIVRÉ	ÉTAT VÉRIFIÉ
Qualification des prospects	Formulaire, persistance, analyse IA, score, intérêt, sujet, résumé et prochaine action.	**Opérationnel**
Conversation IA	Trois scénarios jouables : suivi de colis, cabinet dentaire et garage.	**Opérationnel**
Rendez-vous	Consultation des disponibilités et création d'un événement Google Calendar depuis l'agent.	**Testé en réel**
Relances	Génération manuelle ou quotidienne, message personnalisé conservé en attente de validation.	**Opérationnel**
Gestion de stock	Création, modification, archivage, entrée/sortie atomique et prévention du stock négatif.	**Opérationnel**

> **Principe directeur**
>
> Le navigateur ne contacte jamais directement Supabase avec un rôle privilégié, n8n, Gmail ou Google Calendar. Toutes les actions sensibles passent par le backend Next.js.

02 — VUE D’ENSEMBLE

Architecture applicative

L’architecture reste volontairement simple : un frontend et un backend regroupés dans Next.js, une base PostgreSQL managée par Supabase et une couche d’automatisation n8n. Cette séparation donne à chaque composant une responsabilité claire.



Présentation

Next.js 16, React 19, Tailwind CSS 4, Tabler Icons et Recharts.

Application

Route Handlers, services métier, schémas Zod et contrats d’erreurs stables.

Persistance

Supabase/PostgreSQL, migrations versionnées, contraintes et fonction SQL atomique.

Responsabilités par composant

COMPOSANT	RESPONSABILITÉ	NE FAIT PAS
Frontend	Collecter les entrées, afficher l’état et guider l’utilisateur.	Ne contient aucun secret et ne décide pas seul des autorisations.
Backend Next.js	Valider, limiter, appeler les services et normaliser les réponses.	N’expose pas les erreurs internes ni les credentials.
Supabase	Conserver l’état métier, les relations et la traçabilité.	N’est pas directement accessible par les visiteurs avec des droits d’écriture.
n8n	Exécuter les agents IA et appeler Calendar/Gmail.	N’est jamais appelé directement depuis le navigateur.

03 — PARCOURS MÉTIER

Qualification automatique des prospects

Le formulaire ne produit pas seulement une nouvelle ligne CRM. La demande est envoyée à un agent n8n qui rend une qualification structurée, ensuite validée avant persistance.

1

Saisie

Nom, email, entreprise et demande sont contrôlés par un schéma Zod avant toute écriture.

2

Création du prospect

Supabase crée le prospect dans l’organisation Orchestra avec un statut intermédiaire et une trace d’activité.

3

Qualification IA

Le backend appelle le webhook n8n authentifié avec un identifiant de corrélation.

4

Réponse structurée

L’agent retourne intérêt, niveau, sujet, score, confiance, résumé, tags et prochaine action.

5

Mise à jour et traçabilité

Le CRM est enrichi, l’exécution est journalisée et l’interface présente immédiatement le résultat.

Comportement nominal

- ✓ Score borné entre 0 et 100.
- ✓ Niveaux d’intérêt limités à high, medium ou low.
- ✓ Prospect qualifié si l’intérêt est confirmé et le score atteint 50.
- ✓ Résultat n8n revalidé côté serveur avant enregistrement.

Comportement en cas d’échec

- Le prospect reste enregistré.
- L’échec de qualification est tracé.
- L’API répond avec un statut différé et un message compréhensible.
- Aucune erreur interne n’est exposée au navigateur.

04 — AGENT CONVERSATIONNEL

Conversation, calendrier et email

Un workflow n8n V4 centralise les trois scénarios. Il conserve la mémoire de session, interroge les outils autorisés et renvoie une réponse JSON stable au backend.

SCÉNARIO	CAPACITÉS	INTÉGRATIONS
Colis	Valider un numéro, calculer un statut fictif et proposer un récapitulatif.	Outil métier interne, Gmail sur confirmation.
Dentiste	Identifier le soin, proposer des créneaux, confirmer le nom et l'email.	Google Calendar et Gmail.
Garagiste	Identifier l'intervention et le véhicule, proposer puis réserver un créneau.	Google Calendar et Gmail.

1. **Session stable**

   Le frontend fournit un identifiant de session ; la conversation et chaque message sont persistés dans Supabase.

2. **Mémoire n8n**

   L'agent retrouve le contexte récent et évite de redemander les informations déjà fournies.

3. **Outil adapté**

   Selon le scénario, l'agent consulte le statut colis ou les disponibilités réelles du calendrier.

4. **Confirmation explicite**

   Une adresse email ne peut pas être inventée ; le rendez-vous ou l'envoi est précédé d'un récapitulatif et d'un accord.

5. **Réponse normalisée**

   Le backend reçoit `reply`, `sessionId`, `scenario` et `generatedAt`, puis conserve la réponse.

> **Validation réalisée**
>
> Le suivi colis, l'envoi Gmail, la consultation des disponibilités et la création d'un rendez-vous Calendar ont été testés en conditions réelles. L'événement créé pour le test a ensuite été supprimé.

Point de vigilance : avant une exploitation client sans supervision, l'enchaînement automatique « création du rendez-vous puis email » doit être rendu déterministe dans un seul outil n8n et couvert par un test contractuel dédié.

Automatiser sans perdre le contrôle

Relances personnalisées

Une relance peut être générée depuis le dashboard ou par le cron quotidien Vercel.

- Le prospect et son contexte sont relus côté serveur.
- n8n génère des variantes email, InMail et connexion.
- La variante email est stockée avec le statut `pending_approval`.
- Le cron cible au maximum cinq prospects inactifs depuis trois jours.
- Les prospects ayant déjà une relance ouverte sont exclus.

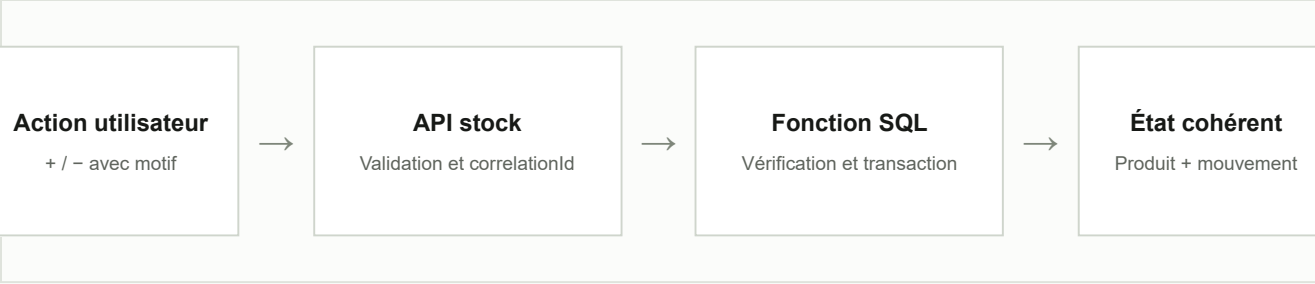
Gestion du stock

Le module représente un commerce informatique et propose un CRUD complet.

- Création avec SKU unique par organisation.
- Modification du nom, seuil, description et prix.
- Archivage logique pour préserver l’historique.
- Entrées et sorties accompagnées d’un motif.
- Signalement des produits sous leur seuil minimal.

Ajustement de stock atomique

L’ajustement n’est pas composé de deux appels indépendants. Une fonction PostgreSQL met à jour la quantité et écrit le mouvement dans une même transaction. Elle refuse une variation nulle, un motif invalide, un produit inactif et toute sortie qui rendrait le stock négatif.



06 — DONNÉES

Modèle Supabase multi-tenant

Les objets applicatifs sont préfixés `orchestra_` afin d'isoler le prototype dans le projet Supabase partagé. Toutes les entités métier portent un `organization_id`.

Organisation

`orchestra_organizations`
`orchestra_organization_members`

CRM

`orchestra_prospects`
`orchestra_activity_logs`

Conversation

`orchestra_conversations`
`orchestra_messages`
`orchestra_appointments`

Automatisation

`orchestra_agents`
`orchestra_workflow_executions`
`orchestra_follow_ups`

Commerce

`orchestra_products`
`orchestra_stock_movements`

Traçabilité

Identifiants de corrélation, statuts, horodatages et résumés non sensibles.

Invariants encodés dans la base

- ✓ Clés étrangères composites empêchant les relations entre deux organisations.
- ✓ RLS activé et forcé sur toutes les tables accessibles par l'application.
- ✓ Aucune politique client accordée par défaut aux rôles navigateur.
- ✓ Unicité des sessions conversationnelles et des SKU dans leur organisation.
- ✓ Index sur les listes croissantes, l'activité, les statuts et les seuils de stock.
- ✓ Clés de corrélation et d'idempotence prévues pour les effets rejouables.

> ### Accès serveur uniquement
>
> La clé Supabase privilégiée reste dans l'environnement Vercel. Les visiteurs passent par les Route Handlers, qui appliquent le périmètre de l'organisation avant chaque lecture ou écriture.

07 — SÉCURITÉ

Frontières et garde-fous

RISQUE	MESURE IMPLÉMENTÉE	EFFET
Entrée invalide	Schémas Zod à l’entrée et à la sortie des webhooks.	Les objets inattendus sont rejetés avant propagation.
Corps excessif	Limite explicite de 16 Ko.	Réduit les abus et le travail inutile du serveur.
Appels répétés	Limites par action et par IP sur une fenêtre d’une minute.	Protège la démo contre les rafales simples.
Fuite de secrets	Credentials n8n et Supabase exclusivement côté serveur.	Le bundle navigateur ne contient aucune clé privilégiée.
Détournement de webhook	Origine n8n contrôlée et credential Header Auth centralisé.	Empêche de rediriger un appel vers un domaine arbitraire.
Appel bloqué	Délais maximum de 30 s, ou 120 s pour le chat.	Les requêtes longues ne restent pas ouvertes indéfiniment.
Erreur interne	Contrat d’erreur stable avec identifiant de corrélation.	Le client obtient une réponse utile sans détail sensible.
Suppression d’historique	Archivage logique des produits.	Les mouvements de stock restent interprétables.

Limites assumées du prototype

- Il n’existe pas encore d’authentification opérateur ni de MFA.
- La limitation de débit est locale à l’instance serverless, pas distribuée.
- Le dashboard utilise une organisation de démonstration unique.
- La relance est générée mais volontairement non envoyée automatiquement.
- Une exploitation commerciale doit ajouter supervision, sauvegardes testées et politique de rétention RGPD.

Une livraison vérifiable

Chaîne de livraison

1. Code versionné dans un dépôt GitHub public.
2. Migrations Supabase conservées avec le projet.
3. Variables sensibles configurées dans Vercel.
4. Déploiement automatique de la branche principale.
5. Vérification du site public après chaque modification.

Contrôles exécutés

- ✓ ESLint sans erreur.
- ✓ TypeScript strict et génération des types Next.js.
- ✓ Build de production Next.js réussi.
- ✓ 8 tests Playwright réussis sur desktop et mobile.
- ✓ Tests live des webhooks, de la base, du stock et des intégrations.

PARCOURS E2E	CE QUI EST CONTRÔLÉ
Dashboard	Chargement, navigation utile et menu mobile.
Prospect	Ouverture du formulaire, saisie et restitution de la qualification.
Conversation	Envoi d’un message et affichage de la réponse agent.
Stock	Ajustement d’une quantité sans rechargement complet de la page.

Exploitation

Le cron Vercel appelle chaque jour la route de relance avec un secret serveur. Le endpoint de santé permet de contrôler la disponibilité générale sans révéler la configuration. Les appels importants transportent un identifiant de corrélation afin de rapprocher une réponse API, une exécution n8n et une trace Supabase.

09 — CONCLUSION

Valeur du prototype

Orchestra démontre la capacité à transformer cinq besoins métier en un système unifié : l'interface collecte et supervise, la base garantit la cohérence, et les agents n8n exécutent les tâches IA ou les intégrations externes. L'ensemble est déployé, testable et documenté.

> ### Ce que ce dossier apporte en plus
>
> Il rend visibles les décisions techniques derrière la démo : séparation des responsabilités, contrats d'API, modèle de données, sécurité, comportement en échec et chemin de passage en production.

Étapes recommandées avant production client

1. Ajouter une authentification opérateur, le MFA et des politiques RLS par organisation.
2. Remplacer la limitation locale par un quota partagé et durable.
3. Regrouper la réservation Calendar et l'email de confirmation dans une action n8n atomique et idempotente.
4. Ajouter des tests contractuels des webhooks et des tests E2E sur un environnement isolé.
5. Mettre en place alertes, métriques, sauvegardes restaurables et règles de rétention.

Document généré à partir de l'état du dépôt au 11 novembre 2026.

Les identifiants internes, URLs privées et valeurs de credentials ont volontairement été exclus.